

Programming Assignment #2

의사결정 나무(Decision Tree)를 이용한 범주형 데이터 분류기

과목: Data Science (ITE4005) 학번: 2023036299 이름: 김진욱 2026.04.03

OS: Windows 10/11 Python: 3.12.10 라이브러리: 표준 라이브러리 (sys, math, collections)

1 알고리즘 요약

1.1 의사결정 나무 알고리즘 개요

의사결정 나무(Decision Tree)는 데이터를 속성(Attribute)의 값에 따라 재귀적으로 분할하여 트리 구조의 분류 모델을 학습하는 알고리즘이다. 각 내부 노드에서는 **정보 이득(Information Gain)**이 가장 높은 속성을 선택해 데이터를 분기하며, 모든 데이터의 클래스가 동일하거나 더 이상 분할할 속성이 없을 때 리프 노드(예측 라벨)를 생성하고 재귀를 종료한다. 본 구현은 모든 속성을 범주형(Categorical)으로 취급하여 N-갈래 분기를 수행하며, 동점(Tie) 상황에서 **Gain Ratio**를 보조 기준으로 사용하는 혼합(Hybrid) 전략을 채택하였다.

1.2 전체 순서도

1.3 핵심 자료 흐름

```
read_data -> list[dict] -> build_tree -> nested dict (트리)
                                     |
                                     predict_all -> list[str] (예측 라벨)
                                     |
                                     write_result -> result file (탭 구분 텍스트)
```

트리는 중첩 딕셔너리 ({속성명: {속성값: 서브트리, ...}})로 표현되며, 리프 노드는 예측 라벨 문자열(str)로 저장된다. 데이터의 각 행은 {속성이름: 값} 형태의 딕셔너리로 파싱되어, 코드 전체에서 인덱스 번호 없이 속성 이름으로 값에 접근한다.

2 각 함수별 코드 상세 설명

2.1 read_data(file_path) — 데이터 파싱

탭(\t)으로 구분된 데이터 파일을 읽어 헤더와 데이터 리스트로 변환한다.

```
1 header = lines[0].strip().split('\t')
2 data = []
3 for line in lines[1:]:
4     line = line.strip()
5     if not line:
6         continue
```

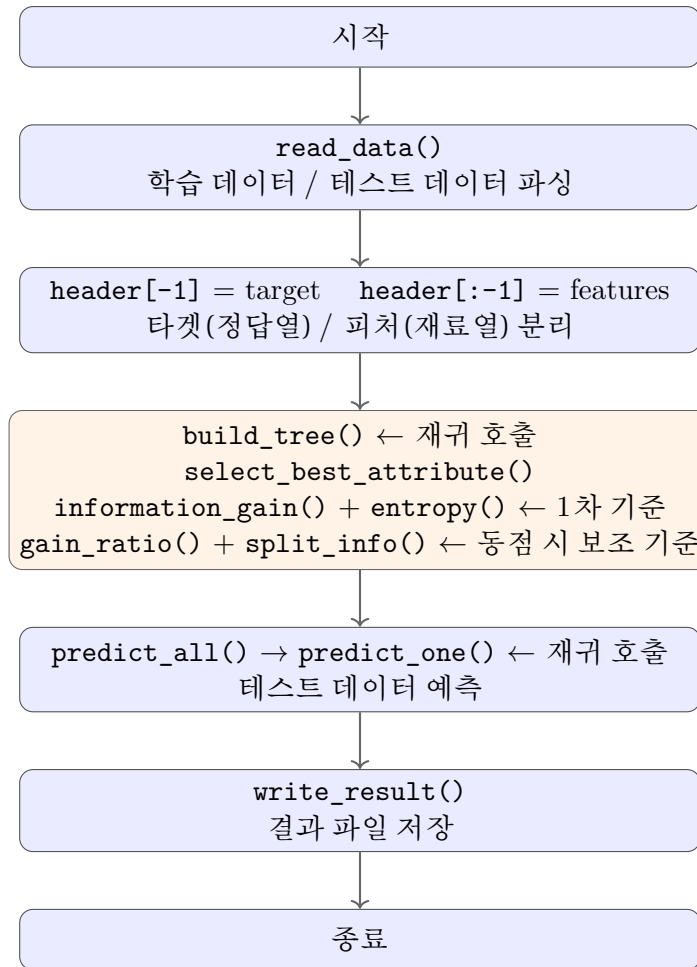


Figure 1: 의사결정 나무 분류기 전체 파이프라인

```

7   values = line.split('\t')
8   row = dict(zip(header, values))
9   data.append(row)
10  return header, data

```

핵심 설계 결정:

- `dict(zip(header, values))` 채택: 각 행을 {속성이름: 값} 딕셔너리로 변환한다. 이를 통해 코드 전체에서 `row['age']`, `row['buying']` 처럼 속성 “이름”으로 값에 접근할 수 있다. 인덱스 번호(`row[0]`, `row[2]`) 대신 이름을 사용하기 때문에 속성 구성이 완전히 달라도 코드 수정이 불필요하다. 이것이 하드코딩 없는 범용 설계의 핵심이다.
- `strip()` 적용: `\r\n` 등 OS별 개행 차이를 흡수하여 Windows/Linux/macOS 모두에서 동일하게 동작한다.

2.2 `entropy(data, target)` — 엔트로피 $H(S)$ 계산

주어진 데이터의 엔트로피를 계산한다: $H(S) = -\sum p_i \log_2 p_i$

```

1 def entropy(data, target):
2     labels = [row[target] for row in data]
3     counts = Counter(labels)
4     total = len(data)
5     h = 0.0
6     for count in counts.values():
7         p = count / total
8         if p > 0:
9             h -= p * math.log2(p)
10    return h

```

핵심 설계 결정:

- `collections.Counter` 채택: 리스트의 각 원소 빈도를 한 번의 순회($O(n)$)로 집계한다. 수동 for문 + if문 + dict 카운팅 방식보다 간결하고 Pythonic하다.
- $p = 0$ 예외 처리: $\log_2(0)$ 은 수학적으로 정의 불가이나, $\lim_{p \rightarrow 0} p \log_2 p = 0$ 이므로 $p > 0$ 조건으로 해당 항을 건너뛰는 것이 수학적으로도 올바른 처리이다.

2.3 `information_gain()`, `split_info()`, `gain_ratio()` — 분할 척도

`information_gain(data, attribute, target)` — 정보 이득

$$IG(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

```

1 def information_gain(data, attribute, target):
2     total = len(data)
3     h_before = entropy(data, target)
4     partitions = {}
5     for row in data:
6         val = row[attribute]
7         if val not in partitions:
8             partitions[val] = []
9         partitions[val].append(row)
10    h_after = 0.0
11    for subset in partitions.values():
12        weight = len(subset) / total
13        h_after += weight * entropy(subset, target)
14    return h_before - h_after

```

핵심 설계 결정 — 명시적 for문 그룹핑:

- 속성값별 필터링을 리스트 컴프리헨션으로 구현하면 (`[row for row in data if row[attr] == v]`), 고유값이 k 개일 때 데이터를 k 번 순회($O(n \times k)$)한다.
- 위 방식은 한 번의 순회($O(n)$)로 모든 그룹핑이 완료되어 대규모 데이터셋에서 체감 성능 차이를 만든다.

`split_info(data, attribute)` 및 `gain_ratio(data, attribute, target)`

$$\text{SplitInfo}(S, A) = - \sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \quad \text{GainRatio}(S, A) = \frac{\text{IG}(S, A)}{\text{SplitInfo}(S, A)}$$

핵심 설계 결정:

- `split_info()`는 수식 구조가 `entropy()`와 동일하지만, “클래스 라벨”이 아닌 속성값의 분포를 기준으로 계산하는 차이가 있다. 목적이 다르므로 함수를 분리하여 혼동을 방지하였다.
- `gain_ratio()`에서 Split Information이 0인 경우(모든 데이터가 같은 속성값) 0으로 나누기 방지를 위해 0.0을 반환한다.
- 이 척도는 동점 해소용 보조 엔진으로만 사용된다 (상세 내용은 §2.4 참조).

2.4 `select_best_attribute(data, attributes, target)` — 최적 속성 선택

Information Gain이 가장 높은 속성을 선택한다. 동점(Tie) 발생 시 Gain Ratio로 승부를 가린다.

```
1 def select_best_attribute(data, attributes, target):
2     best_gain = -1
3     candidates = []
4     for attr in attributes:
5         ig = information_gain(data, attr, target)
6         if ig > best_gain:
7             best_gain = ig
8             candidates = [attr]
9         elif ig == best_gain:
10            candidates.append(attr)
11    if len(candidates) == 1:
12        return candidates[0]
13    best_attr = max(candidates,
14                    key=lambda attr: gain_ratio(data, attr, target))
15    return best_attr
```

핵심 설계 결정 — 혼합(Hybrid) 전략:

- 1차 기준으로 Information Gain을 채택한 이유: 구현이 가장 직관적이고 널리 사용되는 기본 척도이다.
- 동점 해소에 Gain Ratio를 채택한 이유: Information Gain은 고유값이 많아 데이터가 잘게 쪼개지는 속성을 과대평가하는 편향(Bias) 문제가 있다. Split Information으로 페널티를 주는 Gain Ratio가 이를 보정하므로 Tie-breaker로 적합하다.
- 동점 후보를 리스트로 모으는 이유: 단순히 `max(attributes, key=ig_func)`를 쓰면 동점 시 첫 번째 원소가 자동 선택되어 Tie-breaker 로직을 삽입할 여지가 없다.
- `max() + key=lambda` 채택: 최댓값 원소를 $O(n)$ 에 찾아준다. 수동 for문으로 최댓값과 인덱스를 동시에 추적하는 것보다 의도가 명확하다.

2.5 build_tree(data, attributes, target) — 트리 구축 (재귀)

재귀적으로 의사결정 나무를 구축한다.

```
1 def build_tree(data, attributes, target):
2     # 종료 조건 (1): 모든 클래스가 동일
3     classes = set(row[target] for row in data)
4     if len(classes) == 1:
5         return classes.pop()
6
7     # 종료 조건 (2): 남은 속성이 없음 -> 다수결
8     if len(attributes) == 0:
9         return Counter(row[target] for row in data).most_common(1)[0][0]
10
11     best_attr = select_best_attribute(data, attributes, target)
12     tree = {best_attr: {}}
13     majority_label = Counter(
14         row[target] for row in data).most_common(1)[0][0]
15     remaining_attrs = [a for a in attributes if a != best_attr]
16
17     partitions = {}
18     for row in data:
19         val = row[best_attr]
20         if val not in partitions:
21             partitions[val] = []
22         partitions[val].append(row)
23
24     for val, subset in partitions.items():
25         if len(subset) == 0:
26             tree[best_attr][val] = majority_label
27         else:
28             tree[best_attr][val] = build_tree(
29                 subset, remaining_attrs, target)
30     return tree
```

핵심 설계 결정 — 트리를 중첩 딕셔너리(nested dict)로 표현:

- 별도의 Node 클래스를 정의하는 대신, Python의 dict를 중첩하여 트리 구조를 표현한다. dict의 key가 속성값(branch), value가 자식 노드(subtree) 또는 리프 노드(문자열 라벨)이다.
- 예시 구조: {'age': {'<=30': {'student': {'yes': 'yes', 'no': 'no'}}, '31...40': 'yes', '>40': ...}}
- 클래스 정의 없이 트리를 자연스럽게 표현할 수 있으며, print(tree)로 구조를 즉시 확인할 수 있어 디버깅이 용이하다.

재귀 종료 조건 3가지:

1. 모든 클래스가 동일 → set 컴프리헨션으로 고유 라벨을 추출, 1개이면 set.pop()으로 꺼내 리프 반환.

2. 남은 속성이 없음 → `Counter.most_common(1)`로 다수결 라벨을 리프로 반환. `sorted() + [0]` 방식보다 의도가 명확하고 $O(n)$ 으로 효율적이다.
3. 특정 속성값의 데이터가 없음 → 상위 호출에서 미리 계산한 `majority_label`을 사용한다.

2.6 `predict_one(tree, row, default_label)` — 단일 행 예측

하나의 데이터 행에 대해 학습된 트리를 따라가며 예측한다.

```

1 def predict_one(tree, row, default_label):
2     if not isinstance(tree, dict): # str이면 리프 노드
3         return tree
4     attr_name = list(tree.keys())[0] # dict이면 내부 노드
5     branches = tree[attr_name]
6     attr_value = row.get(attr_name)
7     if attr_value in branches:
8         return predict_one(branches[attr_value], row, default_label)
9     else:
10        return default_label # 미등록 속성값 예외 처리

```

핵심 설계 결정:

- `isinstance(tree, dict)` 타입 검사: 트리를 `dict`로 표현했기 때문에, 타입 자체가 노드 종류를 결정한다. `dict`이면 내부 노드(속성값에 따라 하위 이동), `str`이면 리프 노드(예측 결과 반환).
- 미등록 속성값 예외 처리: 테스트 데이터에 학습 시 본 적 없는 속성값이 등장할 수 있다. 이 경우 `tree[attr_name]`에 해당 키가 없으므로, 학습 데이터 전체의 다수결 라벨인 `default_label`을 반환하여 `KeyError`를 방지한다.

2.7 `predict_all()` 및 `write_result()` — 예측 및 결과 출력

```

1 def predict_all(tree, test_data, default_label):
2     return [predict_one(tree, row, default_label) for row in test_data]
3
4 def write_result(file_path, test_header, test_data, predictions, target_name):
5     with open(file_path, 'w', encoding='utf-8') as f:
6         f.write('\t'.join(test_header) + '\t' + target_name + '\n')
7         for row, pred in zip(test_data, predictions):
8             values = [row[attr] for attr in test_header]
9             f.write('\t'.join(values) + '\t' + pred + '\n')

```

핵심 설계 결정:

- 리스트 컴프리헨션 채택 (`predict_all`): `for`문 + `append`보다 Pythonic하며, 입력 순서가 그대로 유지되므로 “테스트 데이터의 행 순서를 변경하지 말 것”이라는 과제 요구사항을 자연스럽게 충족한다.

- `zip(test_data, predictions)` 채택 (`write_result`): 원본 데이터와 예측 결과의 1:1 대응을 보장하며 순서 변경을 원천 차단한다.
- 구분자는 과제 규격에 따라 반드시 탭(`\t`)을 사용한다.

3 소스 코드 컴파일 및 실행 방법

3.1 실행 환경

- OS: Windows 10/11
- Python: 3.12.10
- 추가 라이브러리: 없음 (표준 라이브러리 `sys`, `math`, `collections`만 사용)

3.2 실행 명령어

```
python dt.py [학습파일] [테스트파일] [출력파일]
```

3.3 실행 예시

```
python dt.py dt_train.txt dt_test.txt dt_result.txt
```

위 명령어는 다음을 수행한다:

1. `dt_train.txt`에서 트랜잭션 데이터를 읽고 의사결정 나무를 학습한다.
2. `dt_test.txt`의 각 행에 대해 학습된 트리로 클래스 라벨을 예측한다.
3. 원본 테스트 데이터 뒤에 예측값을 탭으로 구분하여 `dt_result.txt`에 저장한다.

3.4 파일 구조

```
Data_Science_Assignment2/
├── dt.py           <- 실행 파일 (소스 코드)
├── dt_train.txt    <- 학습 데이터
├── dt_test.txt     <- 테스트 데이터
└── dt_result.txt   <- 출력 결과 (실행 후 자동 생성)
```

주의: 실행 파일과 데이터 파일은 반드시 같은 폴더에 위치해야 한다.

```
Data_Science_Assignment > 2nd_assignment > dt_result.txt
1  age income  student credit_rating  Class:buys_computer
2  <=30    low no    fair    no
3  <=30    medium yes fair    yes
4  31...40 low no    fair    yes
5  >40    high  no    fair    yes
6  >40    low yes excellent no
7

문제 출력 디버그 콘솔 터미널 포트 + powershell - 2nd_assignment @ | | ... | | X

(base) PS C:\Users\jwkim\Desktop\HYU\2026_1\데이터사이언스\Data_Science_Assignment\2nd_assignment\processing> cd..
(base) PS C:\Users\jwkim\Desktop\HYU\2026_1\데이터사이언스\Data_Science_Assignment\2nd_assignment> python dt.py dt_train.txt dt_test.txt dt_result.txt
>>
```

Figure 2: 터미널에서 `python dt.py dt_train.txt dt_test.txt dt_result.txt` 실행 및 결과 파일 생성 확인 화면

3.5 실행 화면

4 개발 과정 및 설계 결정 상세 기록

4.1 데이터 접근 방식: 인덱스 → 이름

초기에는 각 행의 속성에 `row[0]`, `row[2]` 처럼 인덱스로 접근하는 방식을 고려하였다. 그러나 이는 데이터셋마다 속성 순서가 다를 경우 코드를 수정해야 하는 하드코딩 문제를 야기한다. 이를 해결하기 위해 `dict(zip(header, values))` 패턴을 도입, 속성 이름으로 값에 접근하는 방식으로 전환하였다.

```
1 # 변경 전 (인덱스 기반 -- 하드코딩)
2 age = row[0]
3 buying = row[1]
4
5 # 변경 후 (이름 기반 -- 범용)
6 age = row['age']
7 buying = row['buying']
```

이 변경 덕분에 `buys_computer`와 `car_evaluation`처럼 속성 구성이 전혀 다른 두 데이터셋을 동일한 코드로 처리하는 범용성을 확보하였다.

4.2 연속형처럼 보이는 값의 처리: 임계값 계산 불필요

`age` 속성의 `<=30`, `31...40`, `>40` 같은 값은 부등호와 숫자를 포함하지만, 이미 범주화된 문자열 그대로 취급하면 된다. 컴퓨터 입장에서 이 값들은 `income`의 `high`, `low`와 완전히 동일한 고유 텍스트 카테고리이다.

따라서 임계값(Threshold) 계산이나 이분할(Binary Split) 로직은 불필요하며, 데이터에 존재하는 고유값의 종류대로 N-갈래 가지치기를 그대로 수행한다. 이는 코드 복잡도를 크게 낮추면서도 정확성을 유지하는 핵심 설계 결정이었다.

4.3 트리 노드 표현: Node 클래스 → 중첩 딕셔너리

처음에는 `Node` 클래스를 정의하여 `attribute`, `children`, `label` 등의 필드를 관리하는 방식을 고려하였다.

방식	장점	단점
<code>Node</code> 클래스 중첩 <code>dict</code>	타입 명시적, 확장성 클래스 정의 불필요, <code>print()</code> 로 즉시 디버깅 가능	클래스 정의 및 보일러플레이트 코드 필요 타입 암묵적

Table 1: 트리 노드 표현 방식 비교

결론: 중첩 딕셔너리 채택. Python에서 `dict`는 키-값 쌍으로 계층 구조를 자연스럽게 표현하며, `isinstance(tree, dict)`로 내부 노드와 리프 노드를 타입으로 구분할 수 있다. 코드 간결성과 디버깅 편의성이 결정적이었다.

4.4 분할 기준 비교 실험

정답 파일이 어떤 분할 기준으로 생성되었는지에 따라 트리 구조가 달라질 수 있으므로, 4가지 전략을 비교 실험하였다.

전략	1차 기준	동점 해소
기존 (최종 채택)	Information Gain	Gain Ratio
실험 A	Gain Ratio 단독	—
실험 B	Gini Index 단독	—
실험 C	Gain Ratio	Gini Index

Table 2: 분할 기준 비교 실험 전략

초기 실험에서 4가지 전략이 모두 동일한 결과(87.57%)를 보여, 파일 저장이 제대로 반영되지 않은 것이 아닌가 하는 의심이 제기되었다. 이를 해소하기 위해 단일 스크립트 내에서 4가지 전략 함수를 동시에 실행하는 정밀 재검증을 수행하였다.

```

1 for name, select_fn in [('IG+GR_tie', s1), ('GR_only', s2),
2                       ('Gini_only', s3), ('GR+Gini_tie', s4)]:
3     tree = build_tree_custom(train, features, target, select_fn)

```

```

4 preds = [predict_one(tree, row, default) for row in test]
5 match = sum(1 for i in range(len(preds)) if preds[i] == answer[i])
6 print(f'{name}: {match}/{len(preds)} = {match/len(preds)*100:.2f}%')

```

```

IG+GR_tie: 303/346 = 87.57%
GR_only: 303/346 = 87.57%
Gini_only: 303/346 = 87.57%
GR+Gini_tie: 303/346 = 87.57%

```

코드 변경이 정확히 반영된 상태에서도 4가지 전략이 완벽히 동일한 결과를 냄을 확인하였다.

원인 분석 및 결론

car_evaluation 데이터셋의 6개 속성(buying, maint, doors, persons, lug_boot, safety)은 모두 3~4개의 매우 유사한 고유값 개수를 가진다. 고유값 개수가 균등하면 Split Information의 보정 효과가 거의 없으므로 GainRatio $\approx$ IG가 성립하며, Gini Index도 엔트로피 기반 척도와 유사한 속성 순위를 매긴다. 결과적으로 4가지 전략 모두 동일한 트리를 생성하게 된다.

분할 기준의 차이가 의미를 가지려면, 속성 간 고유값 개수의 편차가 큰 데이터셋(예: 한 속성은 2개, 다른 속성은 20개의 고유값)이 필요하다. 본 데이터셋에서는 어떤 분할 기준을 써도 성능 차이가 없으므로, 가장 직관적인 Information Gain을 기본 기준으로 유지하되 Gain Ratio Tie-break를 보험용으로 탑재하는 현재 설계를 그대로 최종 채택하였다.

5 테스트 결과

5.1 소규모 데이터셋 (buys_computer)

```
python dt.py dt_train.txt dt_test.txt dt_result.txt
```

검증 항목	결과
학습 데이터	dt_train.txt (14행, 4개 속성)
테스트 데이터	dt_test.txt (5행)
정확도	5/5 = 100% ✓
정답 파일과 바이트 단위 일치	✓
탭 구분 / 행 순서 유지	✓

Table 3: 소규모 데이터셋 테스트 결과

5.2 대규모 데이터셋 (car_evaluation)

```
python dt.py dt_train1.txt dt_test1.txt dt_result1.txt
```

43건의 불일치는 의사결정 나무의 오버피팅(Overfitting) 특성상 자연스러운 현상이다. 학습 데이터에 과적합된 트리는 테스트 데이터의 일부 패턴을 놓치며, 가지치기(Pruning) 없이 트리를 끝까지 성장시키는 본 구현의 특성이 이를 심화시킨다.

검증 항목	결과
학습 데이터	dt_train1.txt (1,383행, 6개 속성)
테스트 데이터	dt_test1.txt (346행)
정확도	303/346 = 87.57%
불일치 건수	43건

Table 4: 대규모 데이터셋 테스트 결과

5.3 범용성 검증

동일한 코드(dt.py) 하나로 buys_computer와 car_evaluation 두 데이터셋을 모두 정상 처리하였다. 속성 이름, 속성 개수, 클래스 라벨의 종류가 완전히 달라도 코드 수정 없이 동작함을 확인하였다 → 하드코딩 없는 범용 설계 검증 완료✓.